

This architecture design prioritizes performance, scalability, and developer experience (DX) for a high-intensity data-driven application like the **AI Fitness Coach**.

1. UI Architecture

- **Framework:** Next.js 14+ (App Router) for Server-Side Rendering (SEO/Performance) and API routes.
- **Language:** TypeScript (Strict mode).
- **Styling:** Tailwind CSS (utility-first) + Shadcn/UI (accessible, headless component primitives).
- **Charting:** Recharts (responsive, clean visualization).
- **Animation:** Framer Motion (for smooth onboarding transitions and daily tip reveals).

2. Folder Structure (Feature-based)

```
src/
├── app/                # App Router (Pages, Layouts)
├── components/
│   ├── ui/            # Shadcn base components
│   ├── dashboard/     # Widgets (ProgressChart, DailyTip, WorkoutCard)
│   ├── forms/         # Multi-step Onboarding logic
│   └── shared/        # Layout, Navbar, AuthGuard
├── hooks/             # Custom hooks (useAuth, useWorkout)
├── lib/               # Utility configurations (Supabase, OpenAI)
├── store/             # Zustand stores
├── types/             # TS interface definitions
└── services/         # API abstraction (AI, User Profile, Workout)
```

3. Pages

- **/onboarding:** Multi-step wizard (Physical stats -> Equipment -> Goals).
- **/dashboard:** The hub. Displays daily workout, AI tip, and progress snapshot.
- **/workout/[id]:** Active session tracker (Interactive checkbox/logger).
- **/progress:** Deep dive into weight trends and body measurements.
- **/settings:** Account management, fitness level adjustments, notification preferences.

4. Components

- **WorkoutCard:** Displays exercises with checkable state.
- **ProgressChart:** Responsive line chart using Recharts for weight tracking.

- **`AllInsight`**: A specialized card that handles the "Daily AI Guidance" with an auto-type effect.
- **`OnboardingWizard`**: Manages state across step transitions.
- **`DataEntryForm`**: Strict 24-hour lock-out validation component.

5. Routing

- **Middleware**: Protected routes via NextAuth/Clerk middleware.
- **Conditional Redirects**: If `user.onboardingComplete`` is false, redirect all auth routes to `/onboarding``.

6. State Management

- **Zustand**: Lightweight global state for user profile and local session data (easier than Redux for this complexity level).
- **TanStack Query (React Query)**: Crucial for data fetching. Handles caching of AI responses (avoiding unnecessary API calls) and background refreshing.

7. Forms & Validation

- **React Hook Form**: For performance, avoids re-renders on keystroke.
- **Zod**: Schema definition for all form data.

Business Logic validation: Ensure `weight`` logs can only be submitted once per 24 hours.

8. Authentication Flow

- **Provider**: Supabase Auth or Clerk.
- **Flow**: Email/Password + OAuth (Google/Apple).
- **Security**: JWT-based sessions. RLS (Row Level Security) on the backend to ensure users only ever access their own biometric data.

9. Dashboard Layout

- **Sidebar Navigation**: Links to Dashboard, Workouts, Progress, Settings.
- **Responsive Grid**:

Desktop: 3-column layout (Left: Sidebar, Center: Workout Plan, Right: AI Tip & Stats).

Mobile: Single column stack, tab-based navigation for quick access during gym sessions.

10. Libraries & Tools

- **Core:** Next.js, TypeScript.
- **UI:** Tailwind CSS, Shadcn/UI, Lucide React (Icons).
- **Data/State:** TanStack Query, Zustand, Zod.
- **Visualization:** Recharts.
- **Formatting:** date-fns (for scheduling and logs).
- **AI Integration:** OpenAI SDK (using `gpt-4-turbo` for structured JSON output).

Architect's Notes:

1. **AI Response Handling:** Since AI generation can take up to 5 seconds, use **Optimistic UI updates** or a loading state with "skeleton screens." Never show a white screen.
2. **Performance:** Implement `React.memo` and `useCallback` for the workout log toggles to ensure the app feels instant during a real gym session.
3. **Data Persistence:** Given the "Use at your own risk" policy, implement a client-side acknowledgment modal that must be accepted before the first workout is rendered.
4. **Scaling:** Use **Vercel** for deployment (Edge Functions for API calls to the AI model reduce latency significantly).